

MACHINE LEARNING COURSE · CS-433

Classification

October 6, 2026

Nicolas Flammarion

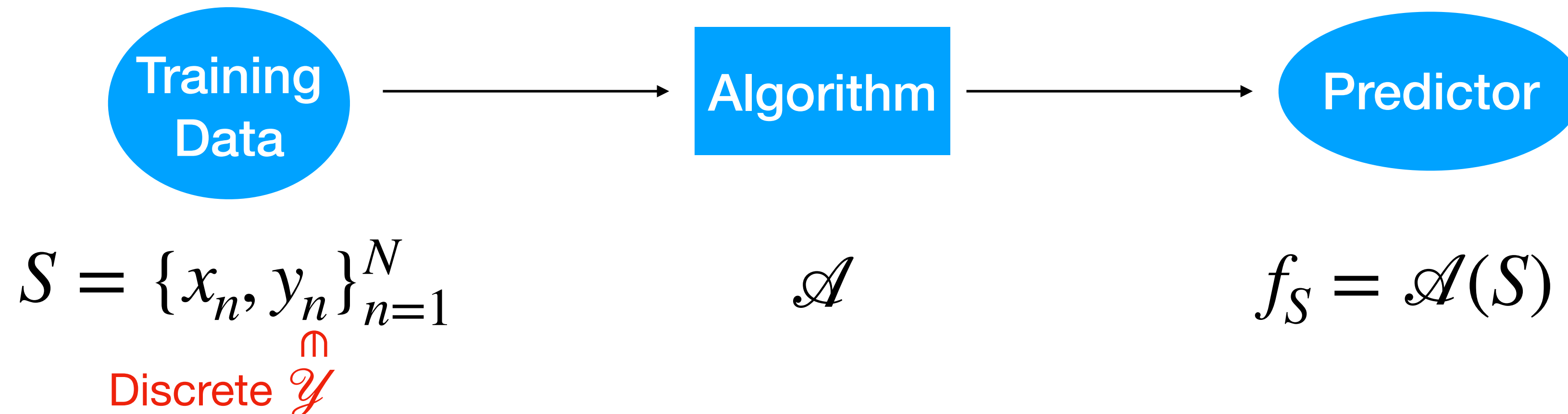
EPFL

Definition of classification

We observe some data $S = \{x_n, y_n\}_{n=1}^N \in \mathcal{X} \times \underbrace{\mathcal{Y}}_{\text{Discrete Set}}$

Goal: given a new observation x , we want to predict its label y

How:



Classification: relates input to a categorical variable

$$(x, y) \in \mathcal{X} \times \underbrace{\mathcal{Y}}_{\text{Discrete Set}}$$

Binary Classification: y can take two values

$y \in \{c_1, c_2\}$ where c_i are the class labels. We often use $\{0, 1\}$ or $\{-1, 1\}$

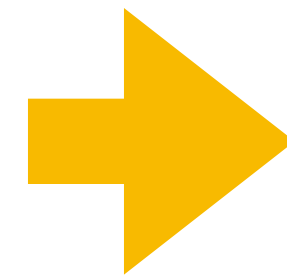
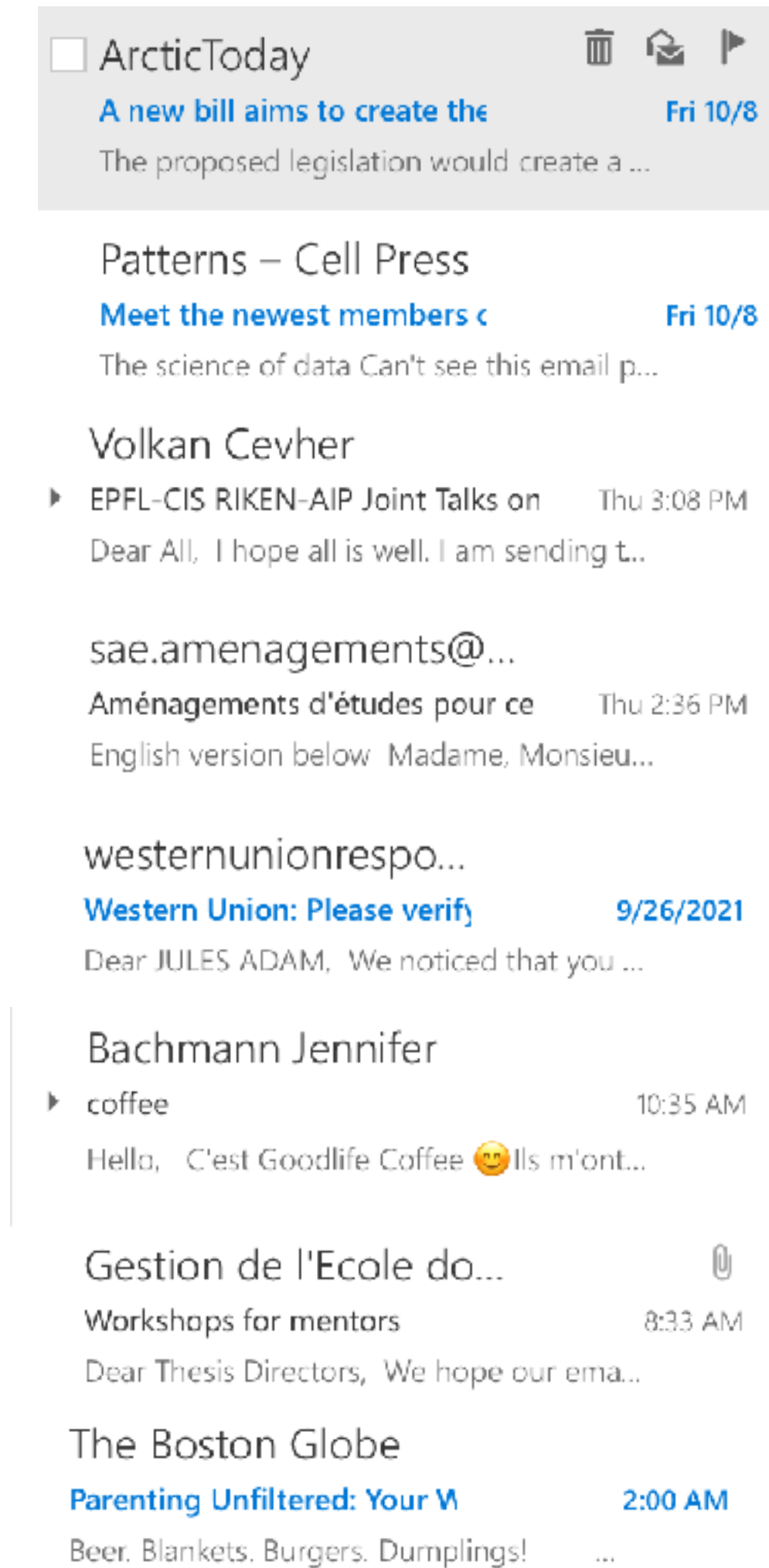
Multi-class classification: y can take more than two values

$y \in \{c_1, \dots, c_K\}$ for a K classes problem. We often use $\{1, \dots, K\}$



no ordering between classes

Spam Detection

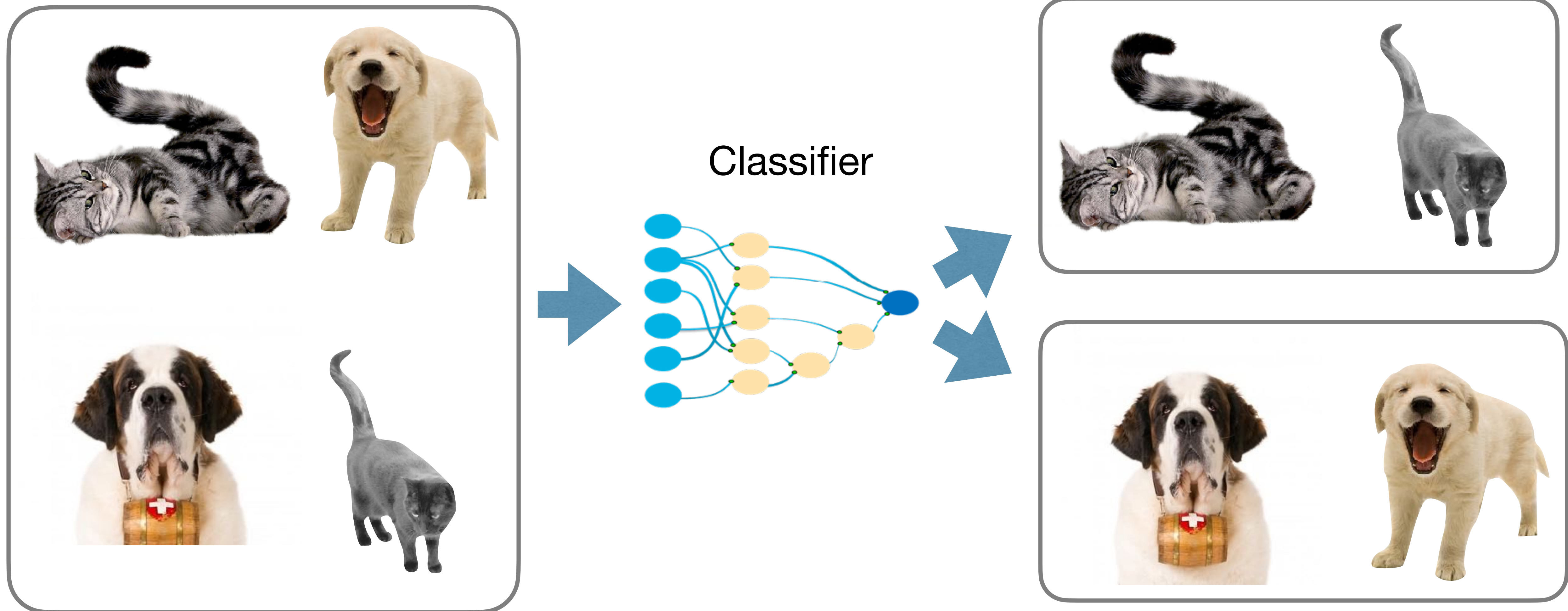


Inbox

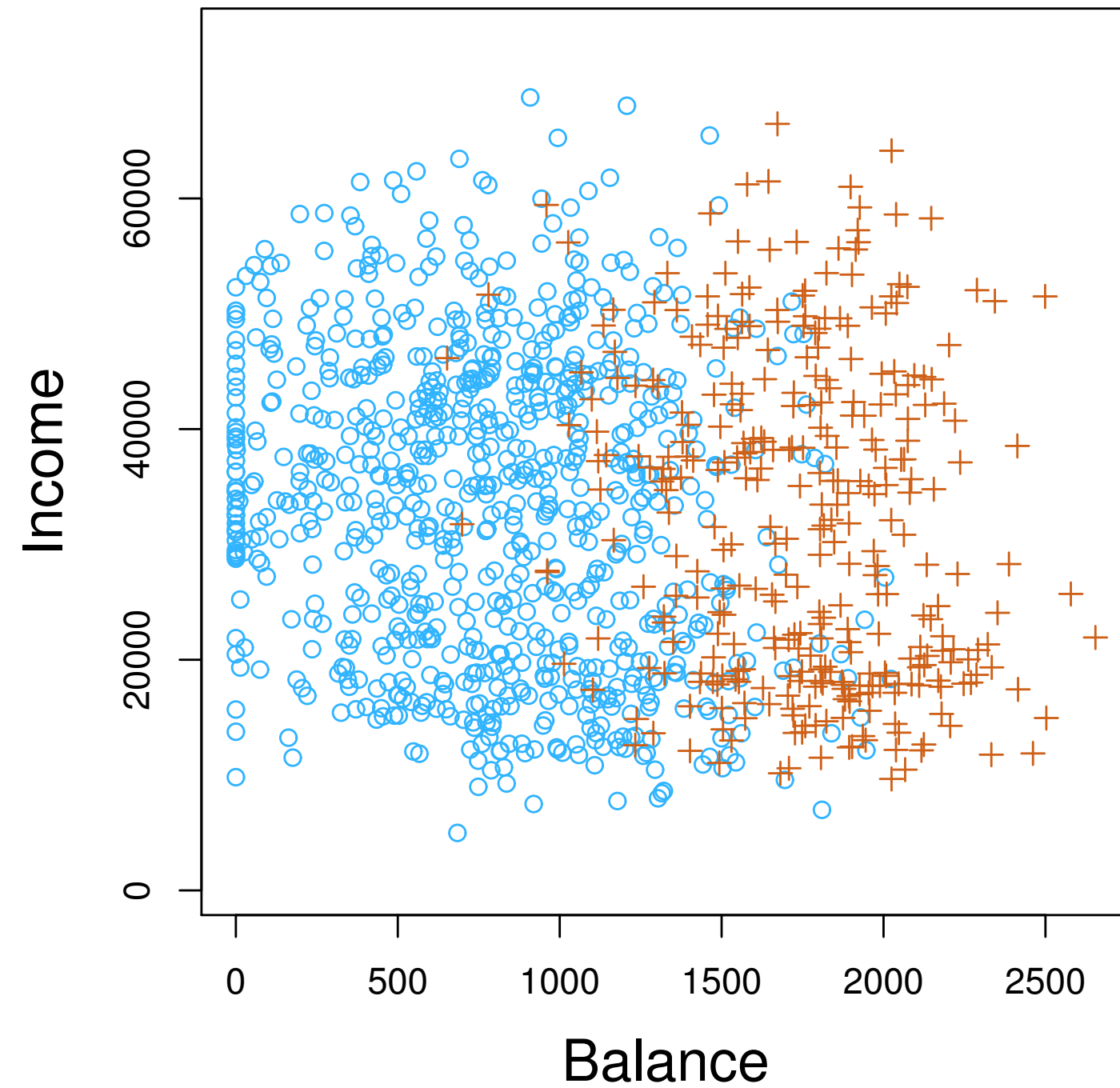


Spam Folder

Image classification



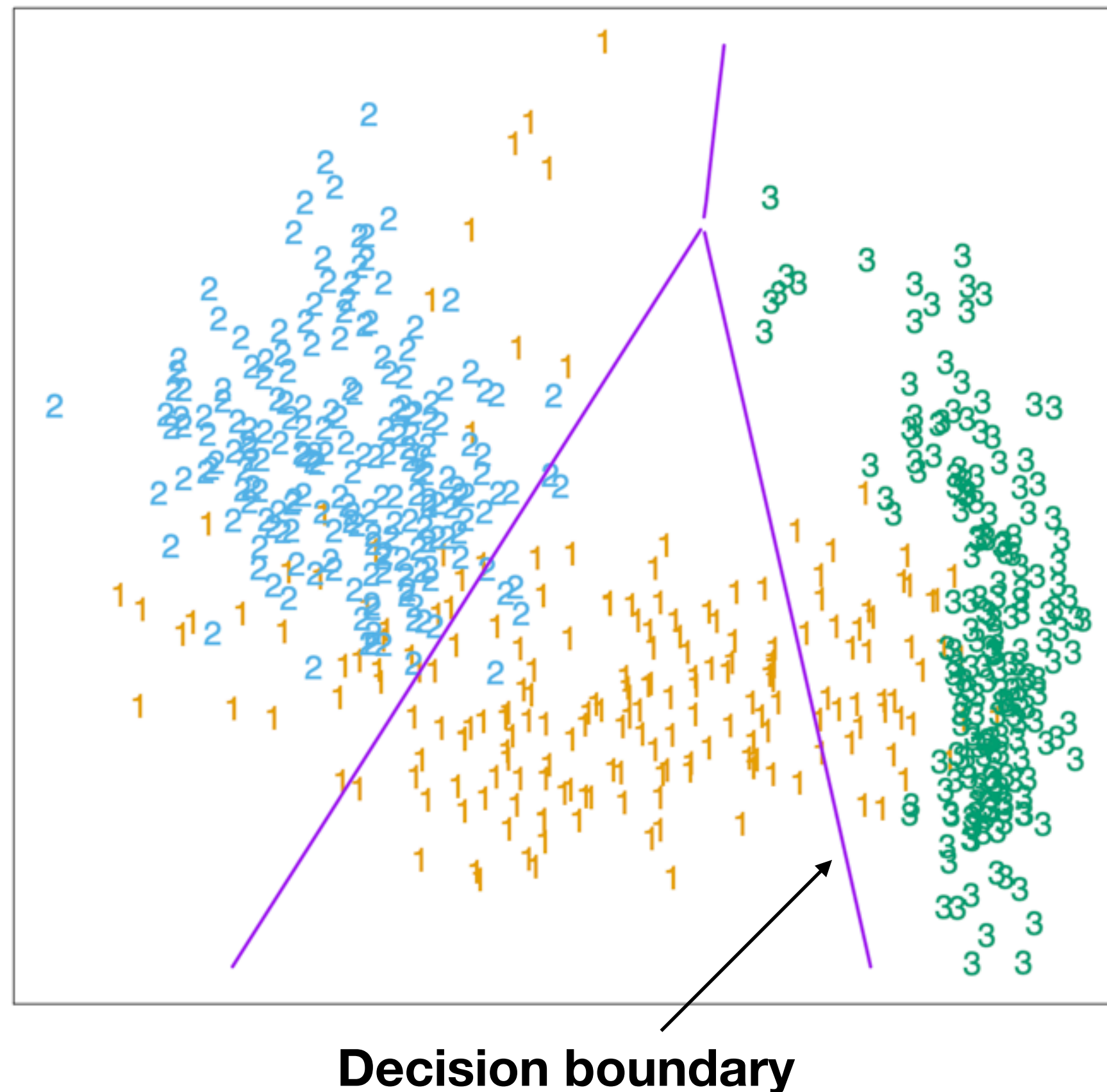
Credit Card Default



- + individual who defaulted on their credit card payments
- individual who did not

Classifier

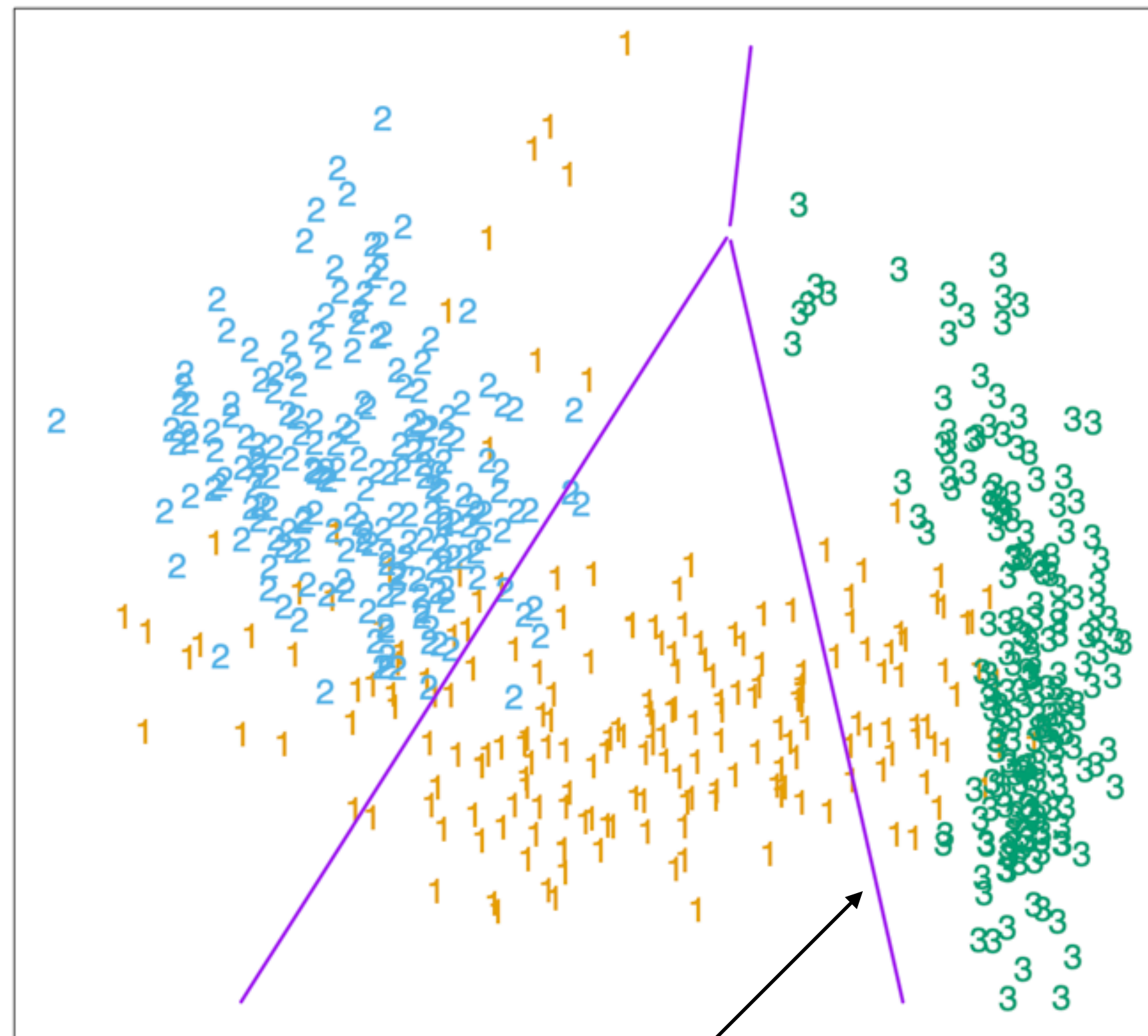
A classifier $f: \mathcal{X} \rightarrow \mathcal{Y}$ divides the input space into a collection of regions belonging to each class



Classifier

A classifier $f: \mathcal{X} \rightarrow \mathcal{Y}$ divides the input space into a collection of regions belonging to each class

It can be linear

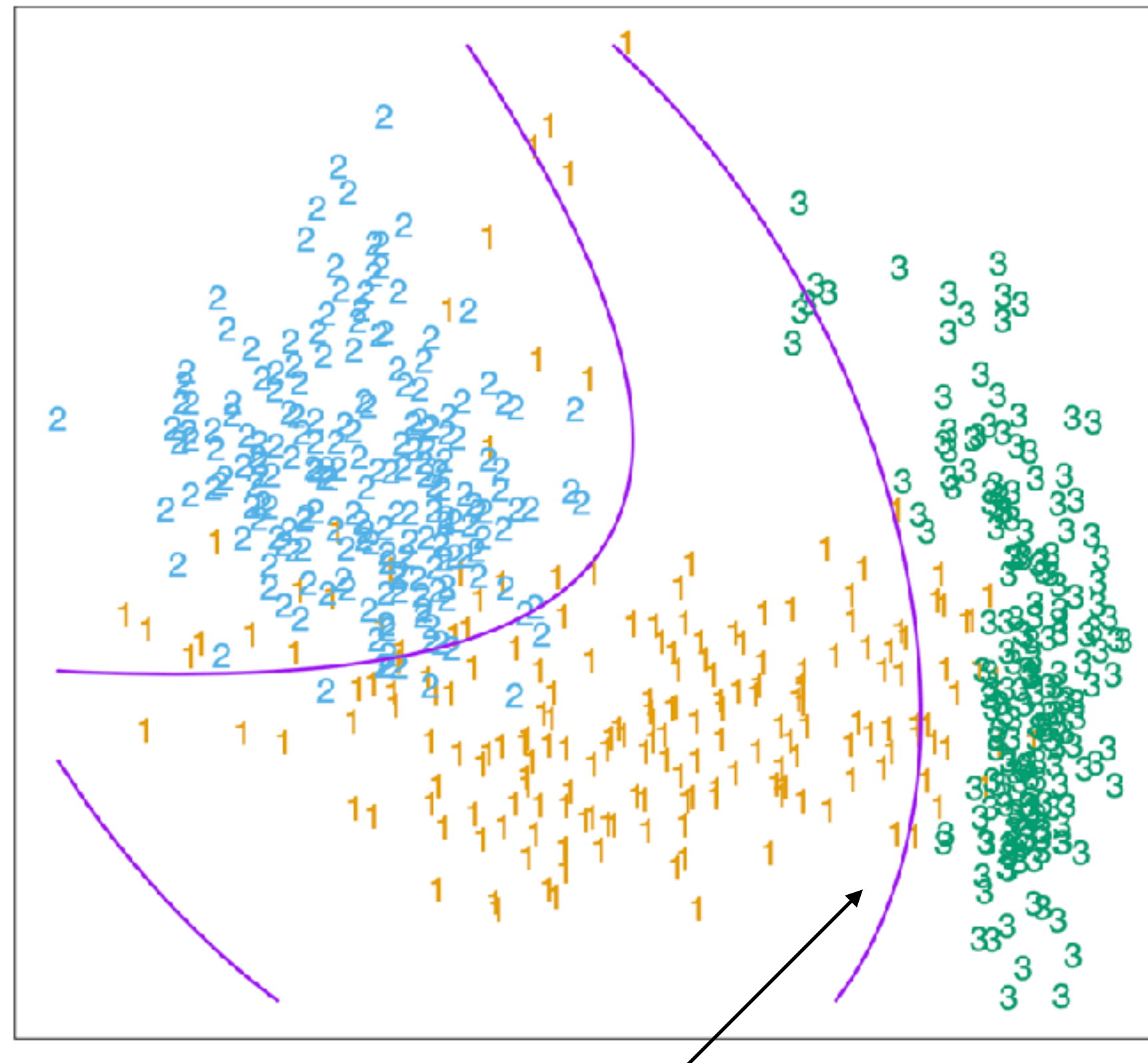


Linear Decision boundary

Classifier

A classifier $f: \mathcal{X} \rightarrow \mathcal{Y}$ divides the input space into a collection of regions belonging to each class

It can also be nonlinear



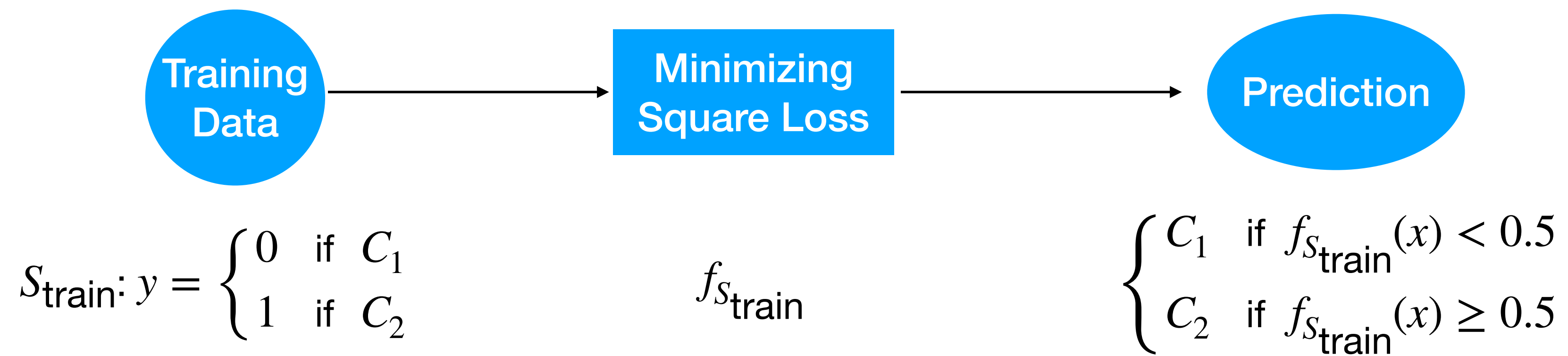
Nonlinear Decision boundary

Classification: a special case of regression?

Classification is a **regression problem** with discrete labels:

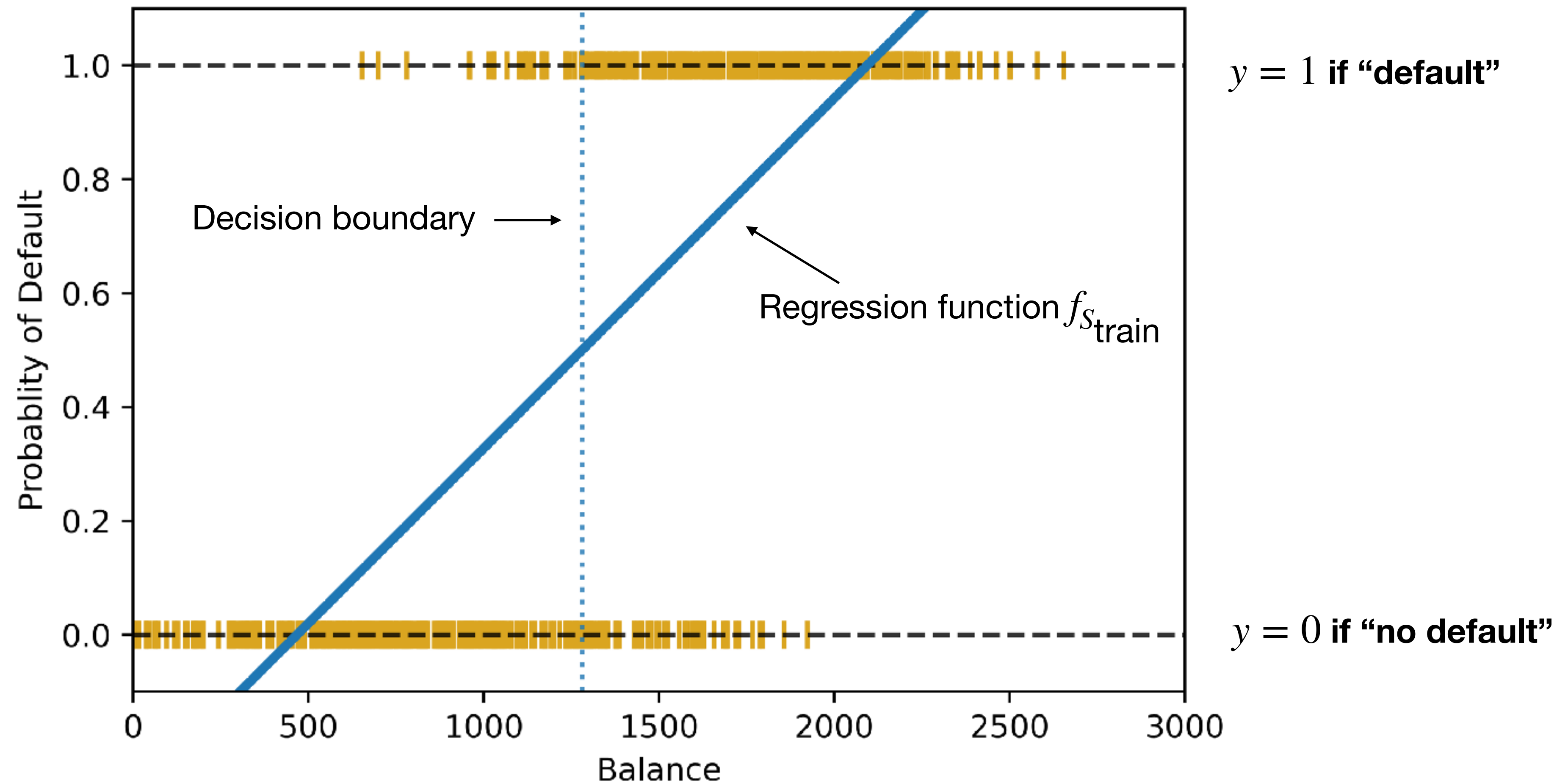
$$(x, y) \in \mathcal{X} \times \{0, 1\} \subset \mathcal{X} \times \mathbb{R}$$

Could we use previously seen regression methods to solve it?



Is it a good idea?

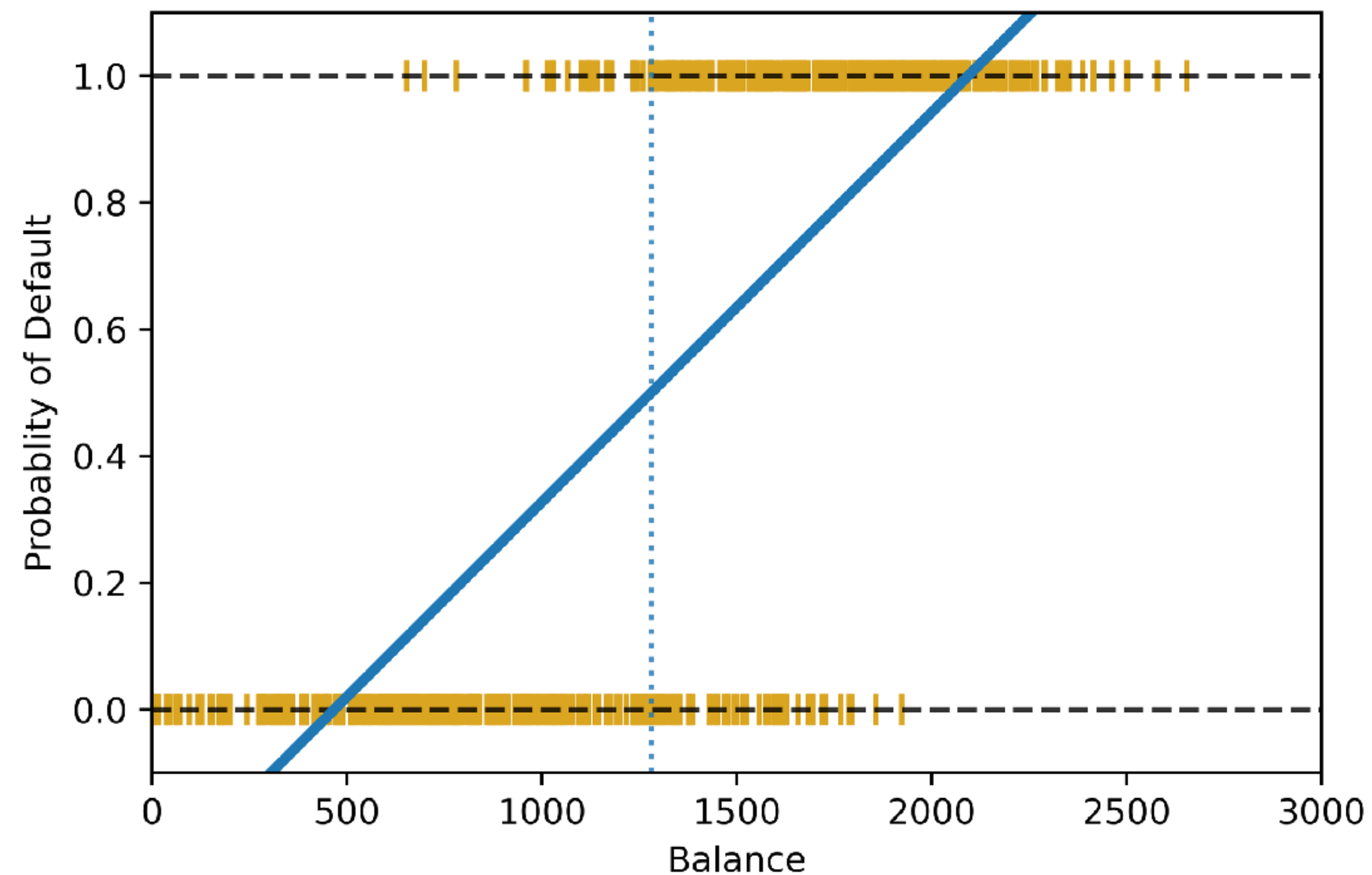
Credit-card default problem:



We label the output as probability for sake of interpretation

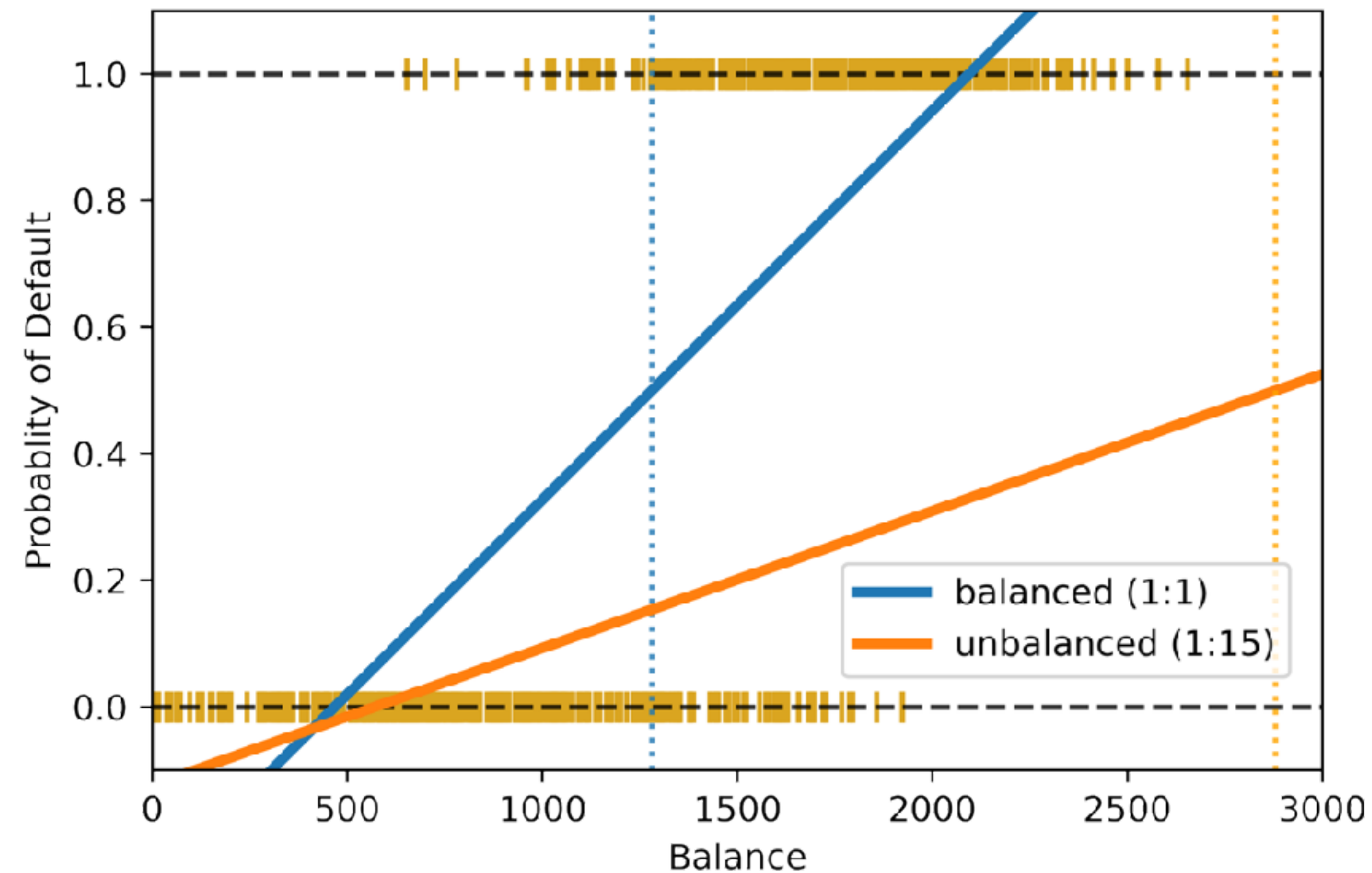
Classification is not just a special form of regression

A. The predicted values are not probabilities (not in $[0,1]$)



Classification is not just a special form of regression

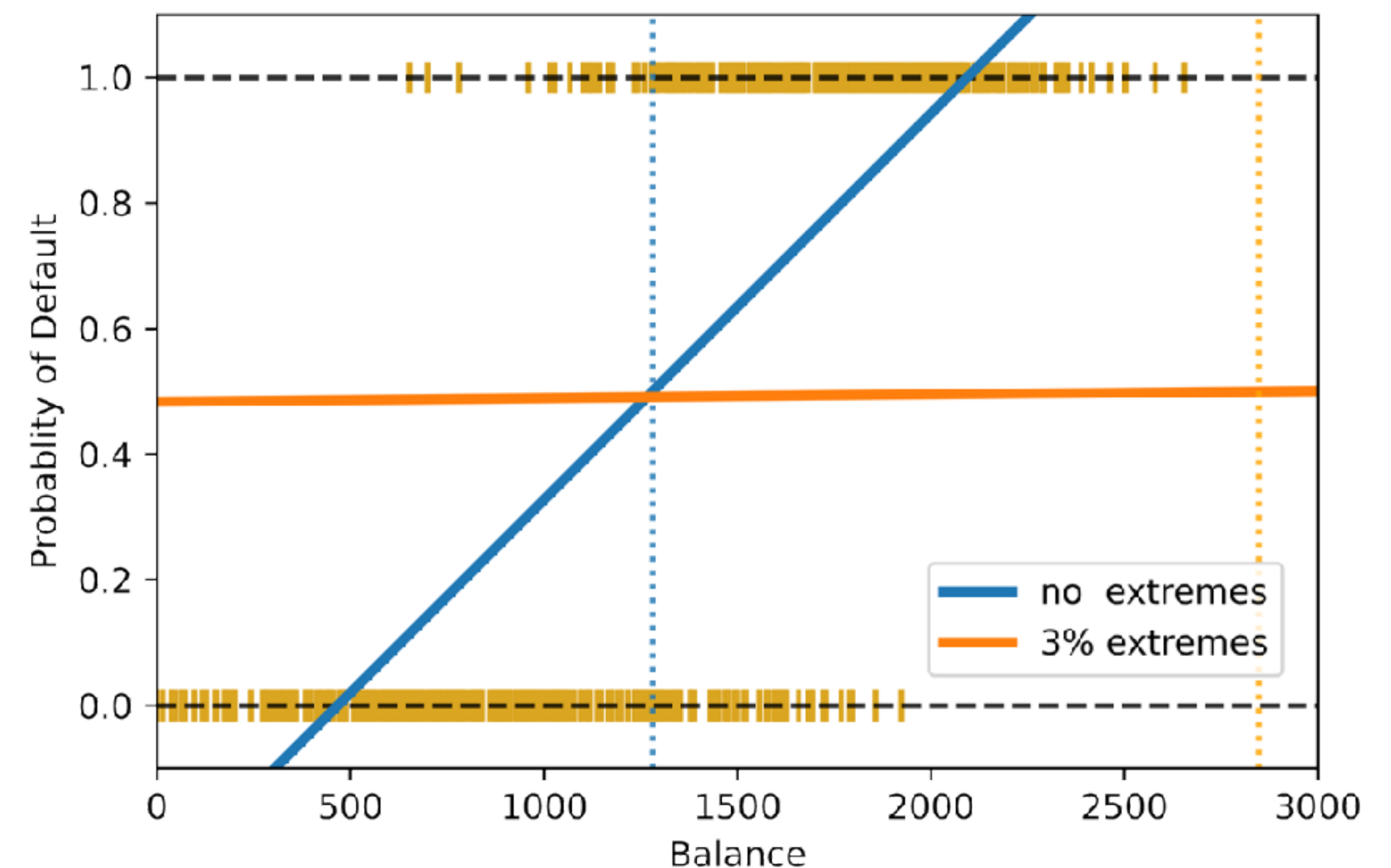
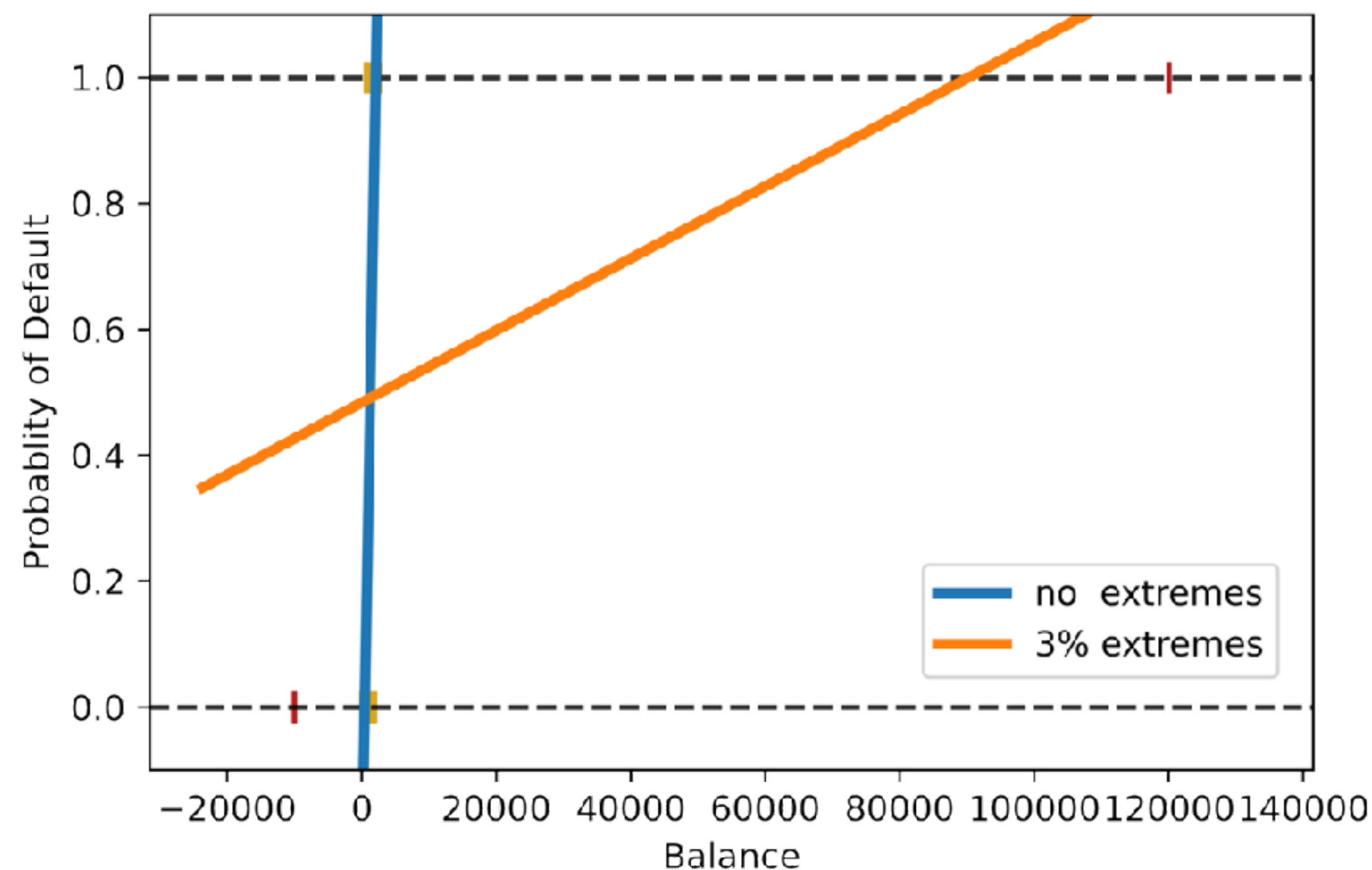
B. Sensitivity to unbalanced data



The position of the line depends crucially on how many points are in each class

Classification is not just a special form of regression

C. Sensitivity to extreme values:



The position of the line depends crucially on where the points lie

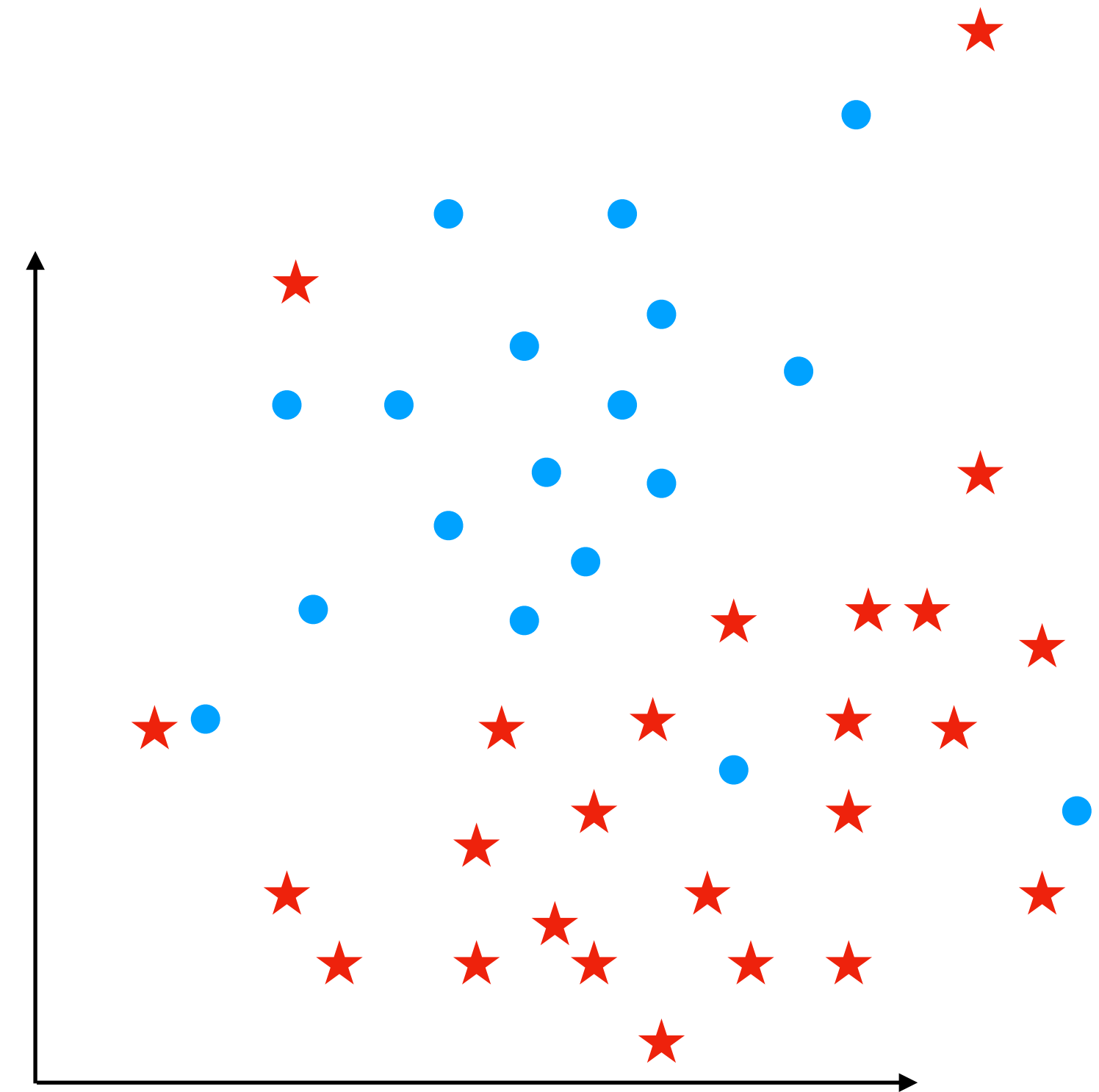
Why: the square loss we used for regression is not suitable for classification

How to perform classification?

- Many approaches have been developed
- We won't cover them in detail today
- Instead, we will provide quick introductions

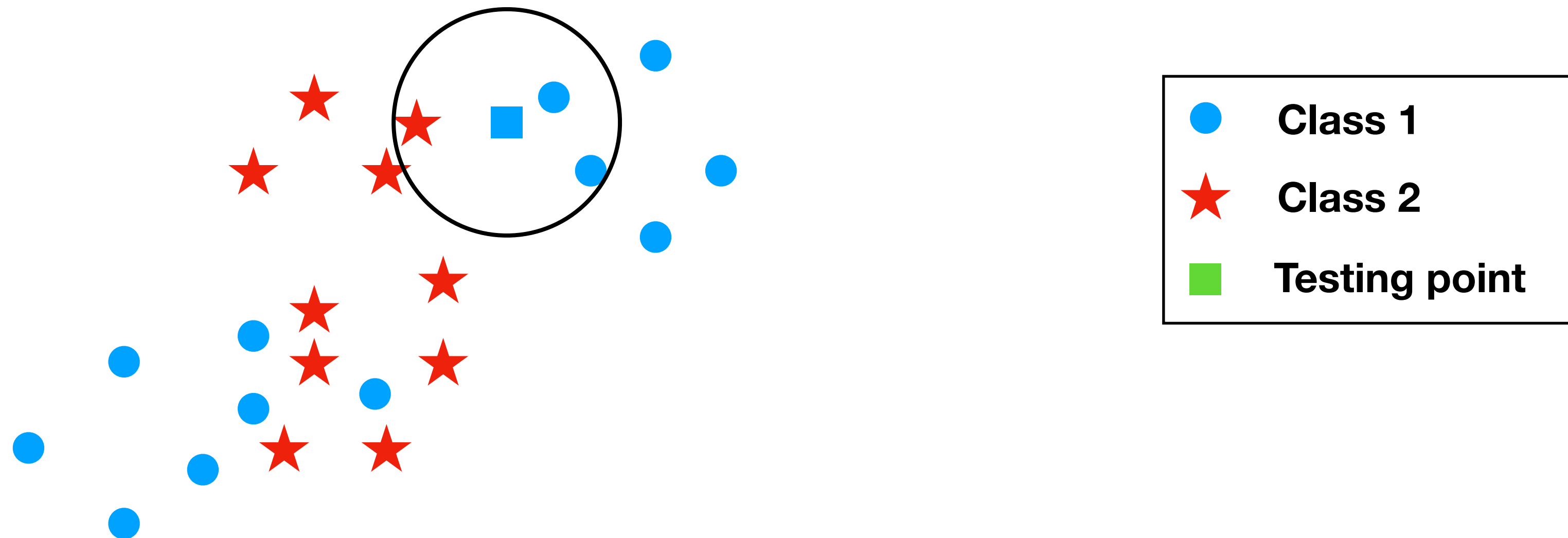
- Fundamental task of classification:

Divide the space into distinct decision regions



k-Nearest Neighbor

Assume that nearby points are likely to have similar labels



A new point x is classified based on the **majority vote of its k-nearest neighbors**

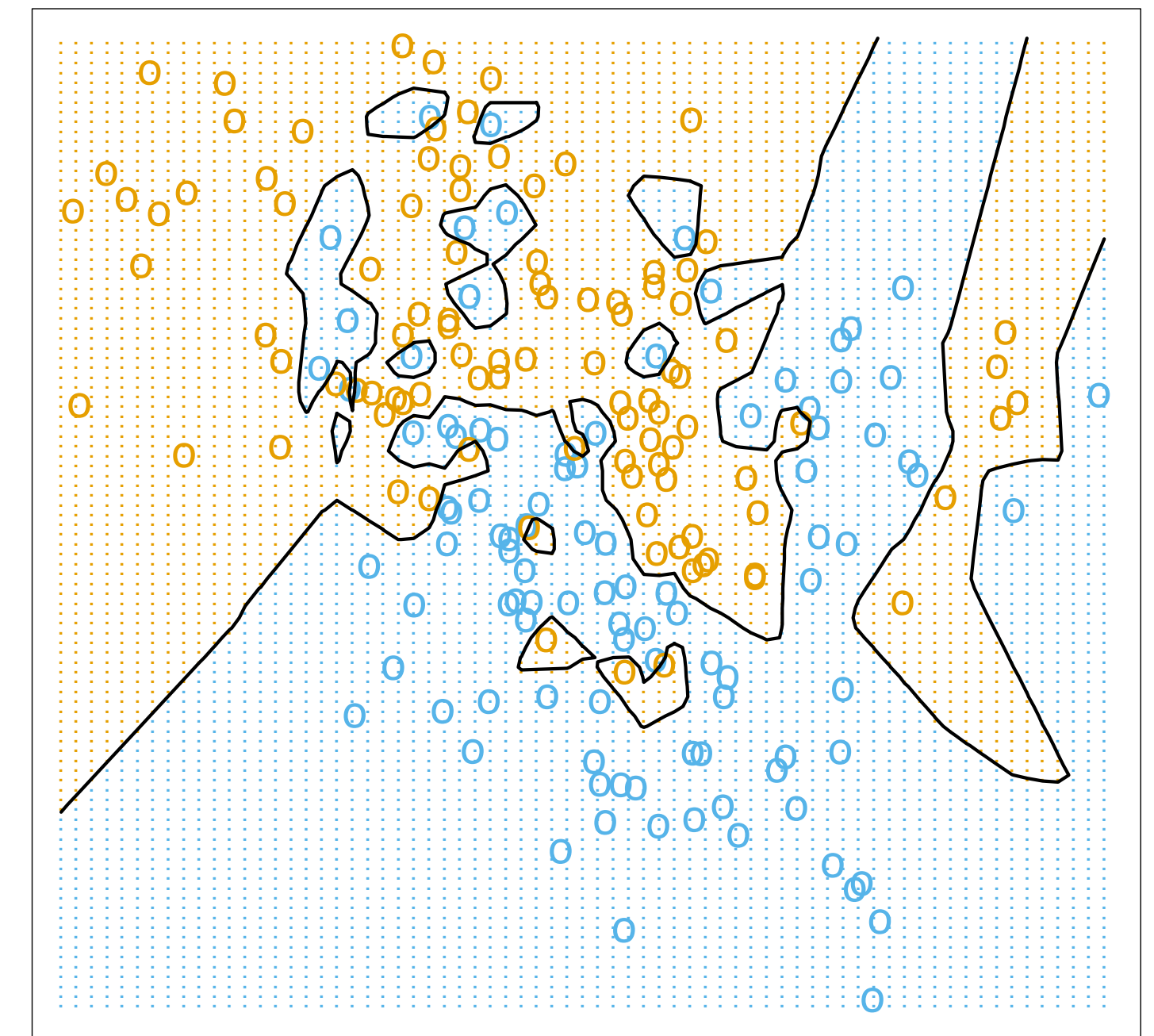
k-Nearest Neighbor

Pros:

- **No optimization** or training
- **Easy** to implement
- Works well in **low dimensions**, allowing for very complex decision boundaries

Cons:

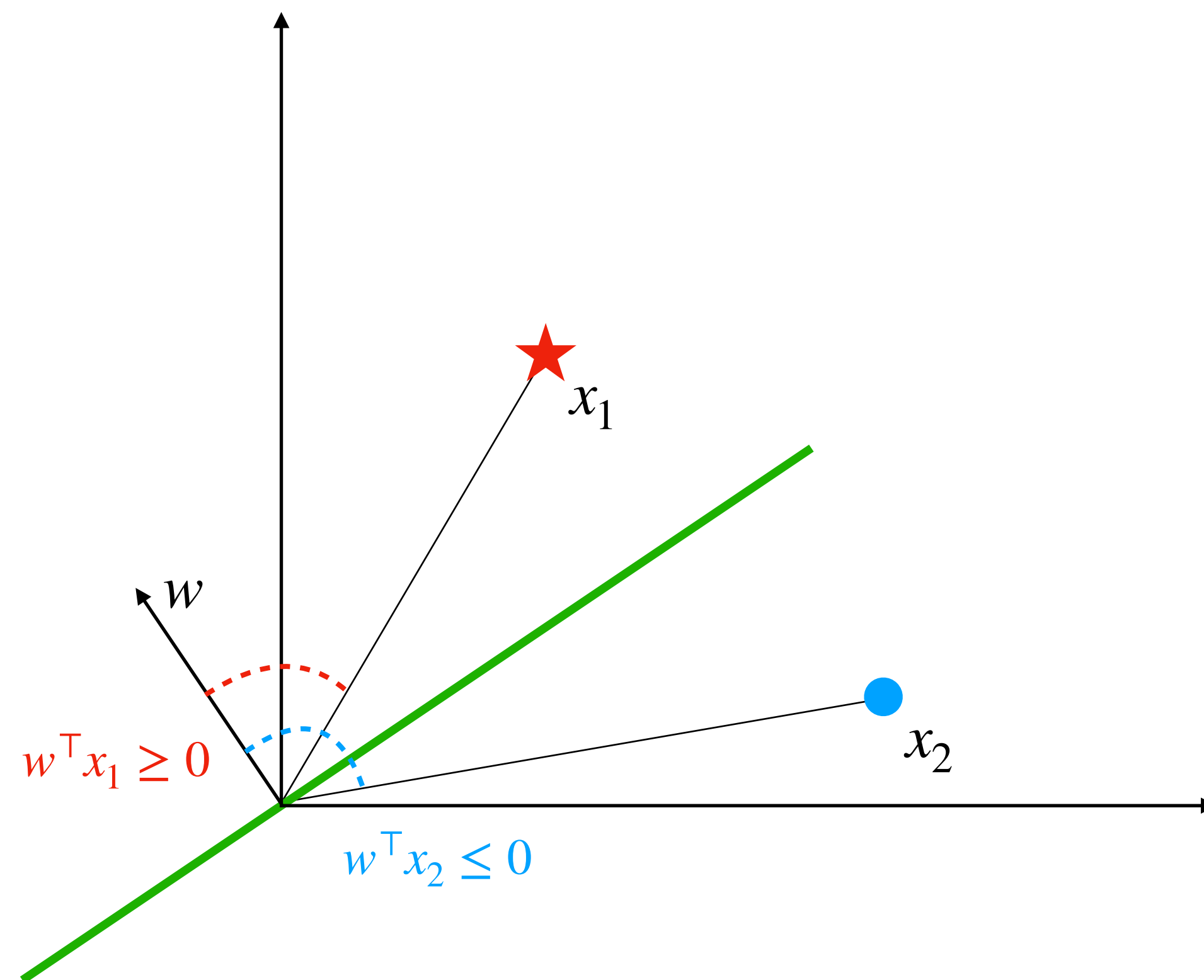
- **Slow** at query time
- Not suitable for high-dimensional data
- Choosing the right local distance is crucial



Linear Decision boundaries

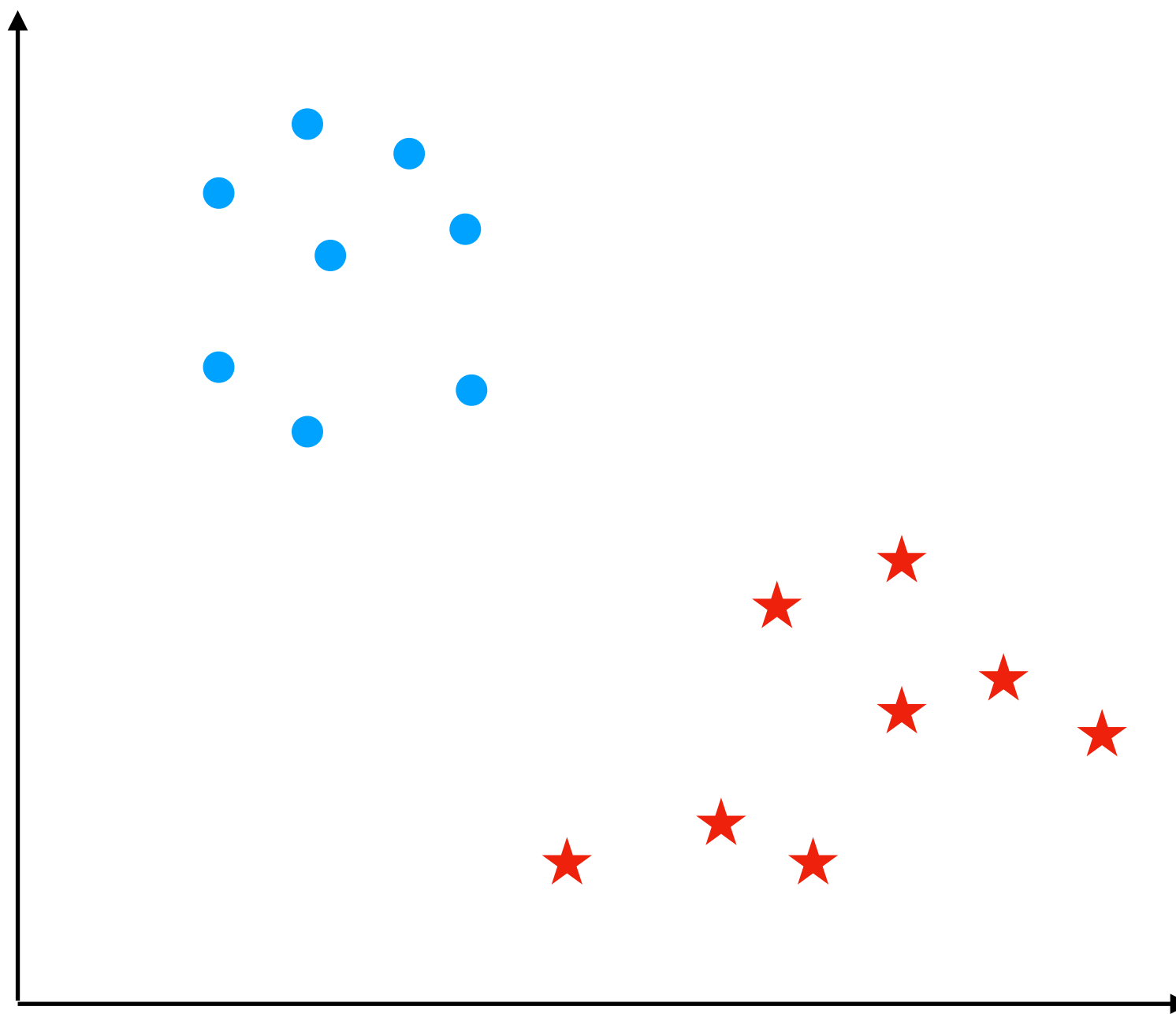
Assume we restrict ourselves to linear decision boundaries (hyperplane):

➡ Prediction: $f(x) = \text{sign}(x^\top w)$



Separating hyperplane

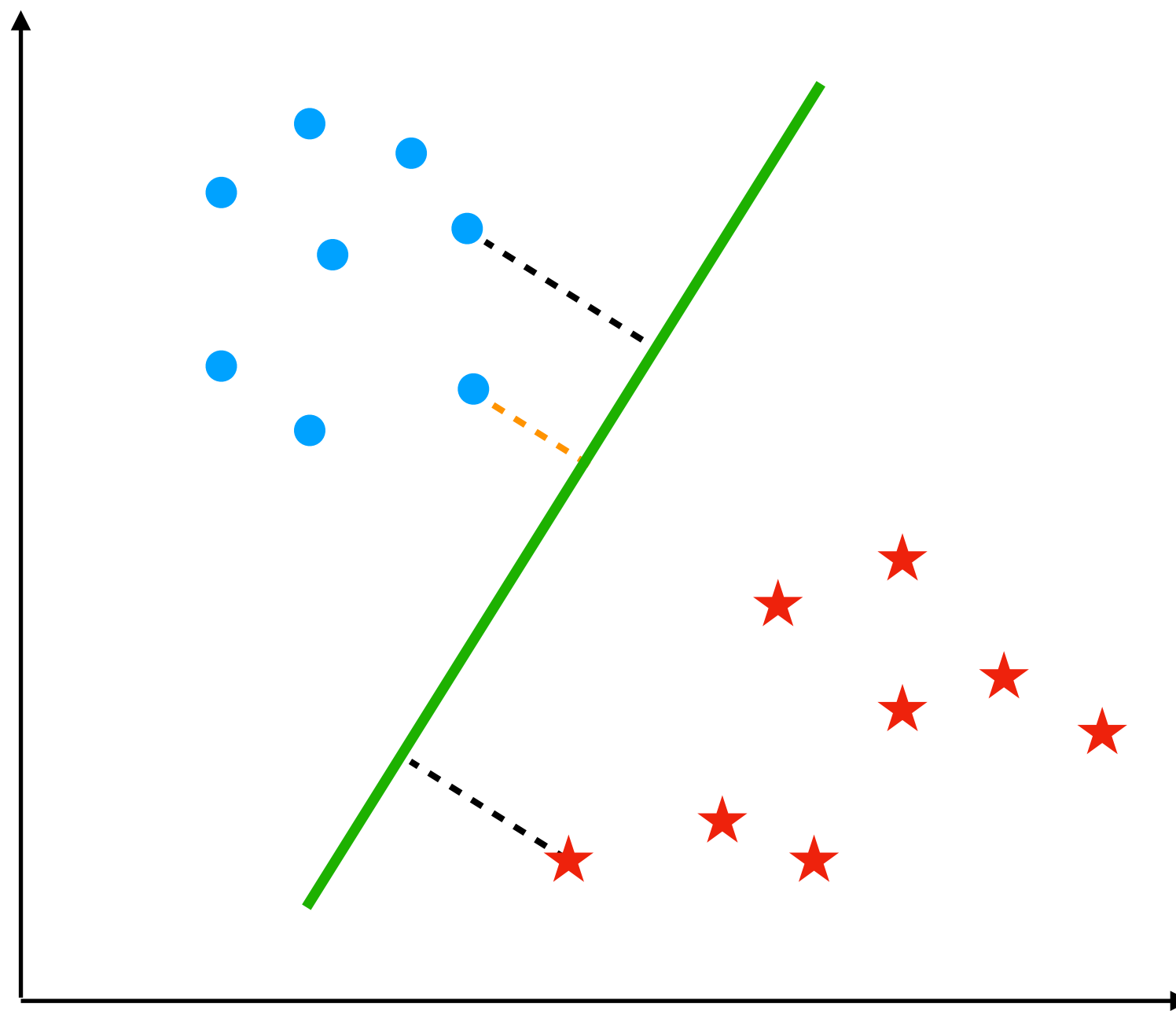
Assume we restrict ourselves to linear decision boundaries (hyperplane):
Assume the data are linearly separable, i.e., a separating hyperplane exists



Which separating hyperplane would you pick?

Margin

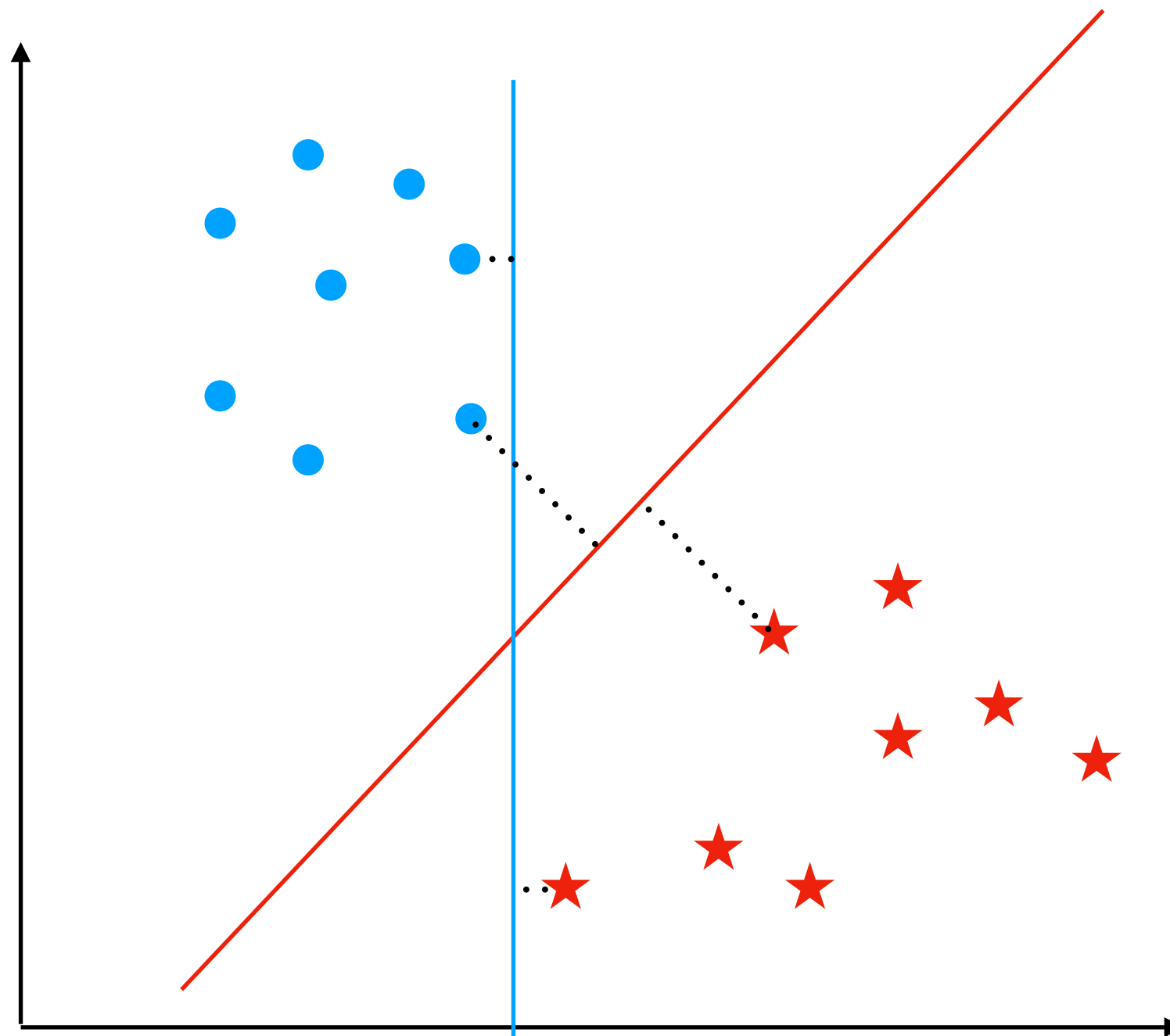
Key concept: The margin is the distance from the hyperplane to the closest point



➡ Take the one with the largest margin!

Max-margin separating hyperplane

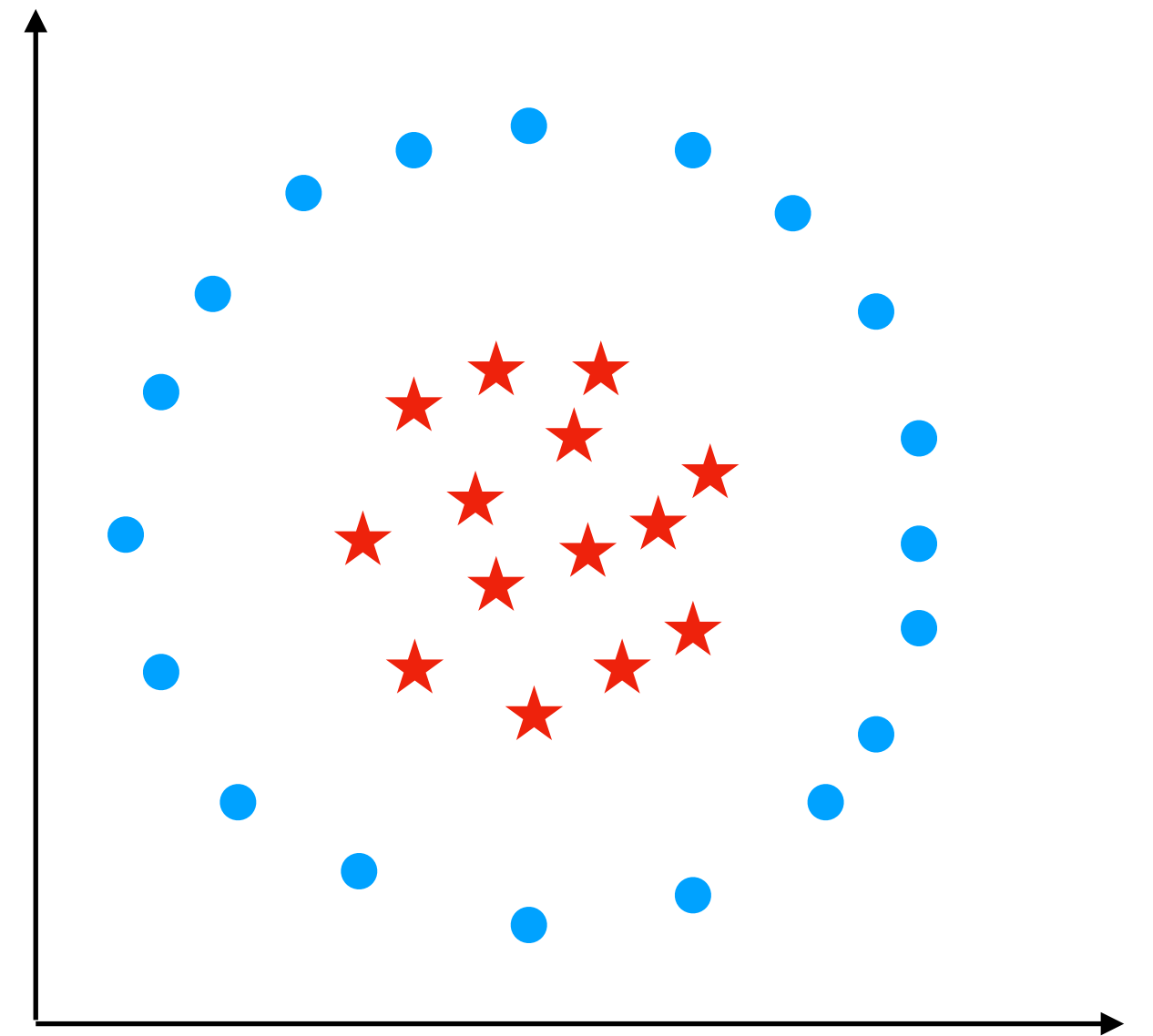
Choose the hyperplane which maximizes the margin



Why: If we slightly change the training set, the number of misclassifications will stay low
➡ It will lead us to support vector machine (SVM) and logistic regression

Nonlinear classifier

- Linear decision boundaries will not always work
- Feature augmentation (x, x^2, x^3, x^4)
- Kernel Method



Formalizing Binary Classification

Setting: $(X, Y) \sim \mathcal{D}$ with ranges $\mathcal{X}, \mathcal{Y} = \{-1, 1\}$

Loss function: (0-1 Loss) $\ell(y, y') = 1_{y \neq y'} = \begin{cases} 1 & \text{if } y \neq y' \\ 0 & \text{if } y = y' \end{cases}$

True risk for the classification:

$$L_{\mathcal{D}}(f) = \mathbb{E}_{\mathcal{D}}[1_{Y \neq f(X)}] = \mathbb{P}_{\mathcal{D}}[Y \neq f(X)]$$

classification error

probability of making an error

Goal: minimize $L_{\mathcal{D}}(f)$

Bayes classifier

What is the **optimal performance**, regardless of the finiteness of the training data?

Def: The classifier $f_* \in \arg \min_f L_{\mathcal{D}}(f)$ is called the **Bayes classifier**

Claim:

$$f_*(x) \in \arg \max_{y \in \{-1, 1\}} \mathbb{P}(Y = y | X = x)$$

Note: Bayes classifier is an *unattainable* gold standard, as we never know the underlying data distribution $\mathcal{D}$ in practice

Proof of the Bayes classifier

Step 1: At each input, choose the label with the smallest conditional error

Define the conditional error $r_x(y) = \mathbb{P}(Y \neq y | X = x) = 1 - \mathbb{P}(Y = y | X = x)$

Choose g_* such that: $g_*(x) \in \arg \max_{y \in \{-1, 1\}} \mathbb{P}(Y = y | X = x)$

Thus, for every classifier f and input x :

$$r_x(f(x)) - r_x(g_*(x)) \geq 0$$

Step 2: Average over the inputs

By the law of total expectation:

$$L_{\mathcal{D}}(f) = \mathbb{E}_{X,Y}[1_{Y \neq f(X)}] = \mathbb{E}_X[\mathbb{E}_{Y|X}[1_{Y \neq f(X)} | X]] = \mathbb{E}_X[\mathbb{P}(Y \neq f(X) | X)] = \mathbb{E}_X[r_X(f(X))]$$

Therefore,

$$L_{\mathcal{D}}(f) - L_{\mathcal{D}}(g_*) = \mathbb{E}_X \left[\underbrace{r_X(f(X)) - r_X(g_*(X))}_{\geq 0 \text{ by Step 1}} \right] \geq 0$$

Thus $g_* \in \arg \min_f L_{\mathcal{D}}(f)$, and we can choose $f_* = g_*$

Two classes of classification algorithms

- **Non-parametric:** approximate the conditional distribution $\mathbb{P}(Y = y | X = x)$ via local averaging
 - ➡ Follow nearest neighbors' decisions (KNN)
- **Parametric:** approximate true distribution $\mathcal{D}$ via training data
 - ➡ Minimize the empirical risk on training data (ERM)

Classification by empirical risk minimization

How: minimize the empirical risk instead of the true risk:

$$\min_{f:\mathcal{X}\rightarrow\mathcal{Y}} L_{\text{train}}(f) := \frac{1}{N} \sum_{n=1}^N 1_{f(x_n)\neq y_n} = \frac{1}{N} \sum_{n=1}^N 1_{y_n f(x_n) \leq 0}$$

Problem: L_{train} is not convex:

1. The set of classifiers is not convex because $\mathcal{Y}$ is discrete
2. The indicator function 1 is not convex because it is not continuous

Convex relaxation of the classification risk

1. Instead of learning $f: \mathcal{X} \rightarrow \mathcal{Y}$, learn $g: \mathcal{X} \rightarrow \mathbb{R}$ in a convex subset of continuous functions $\mathcal{G}$, and predict with $\hat{f}(x) = \text{sign}(g(x))$. The problem becomes

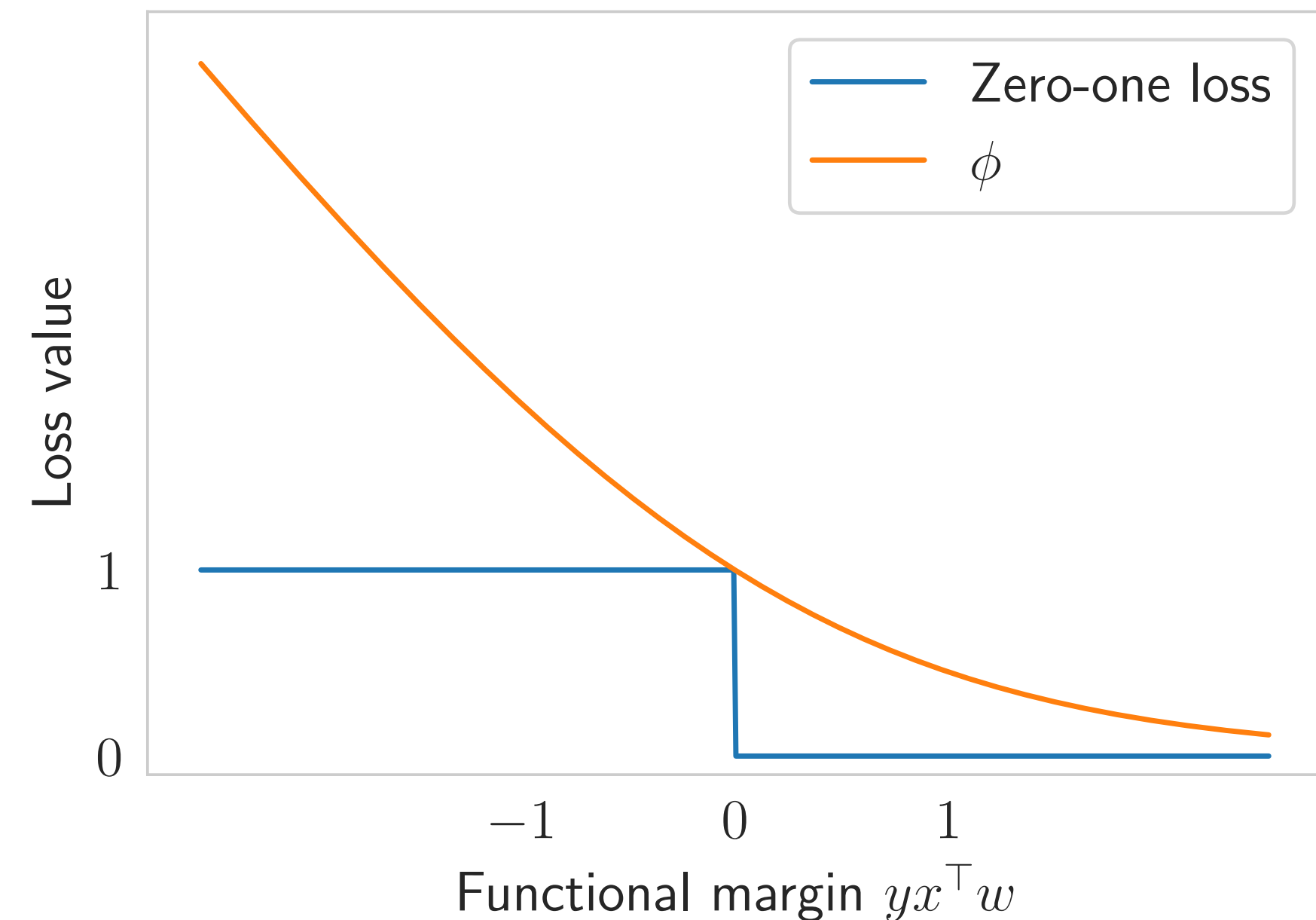
$$\min_{g \in \mathcal{G}} \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{y_n g(x_n) \leq 0}$$

2. Replace the indicator function by a convex $\phi: \mathbb{R} \rightarrow \mathbb{R}$ and minimize

$$\min_{g \in \mathcal{G}} \frac{1}{N} \sum_{n=1}^N \phi(y_n g(x_n))$$

ϕ is a function of the functional margin $y_n g(x_n)$

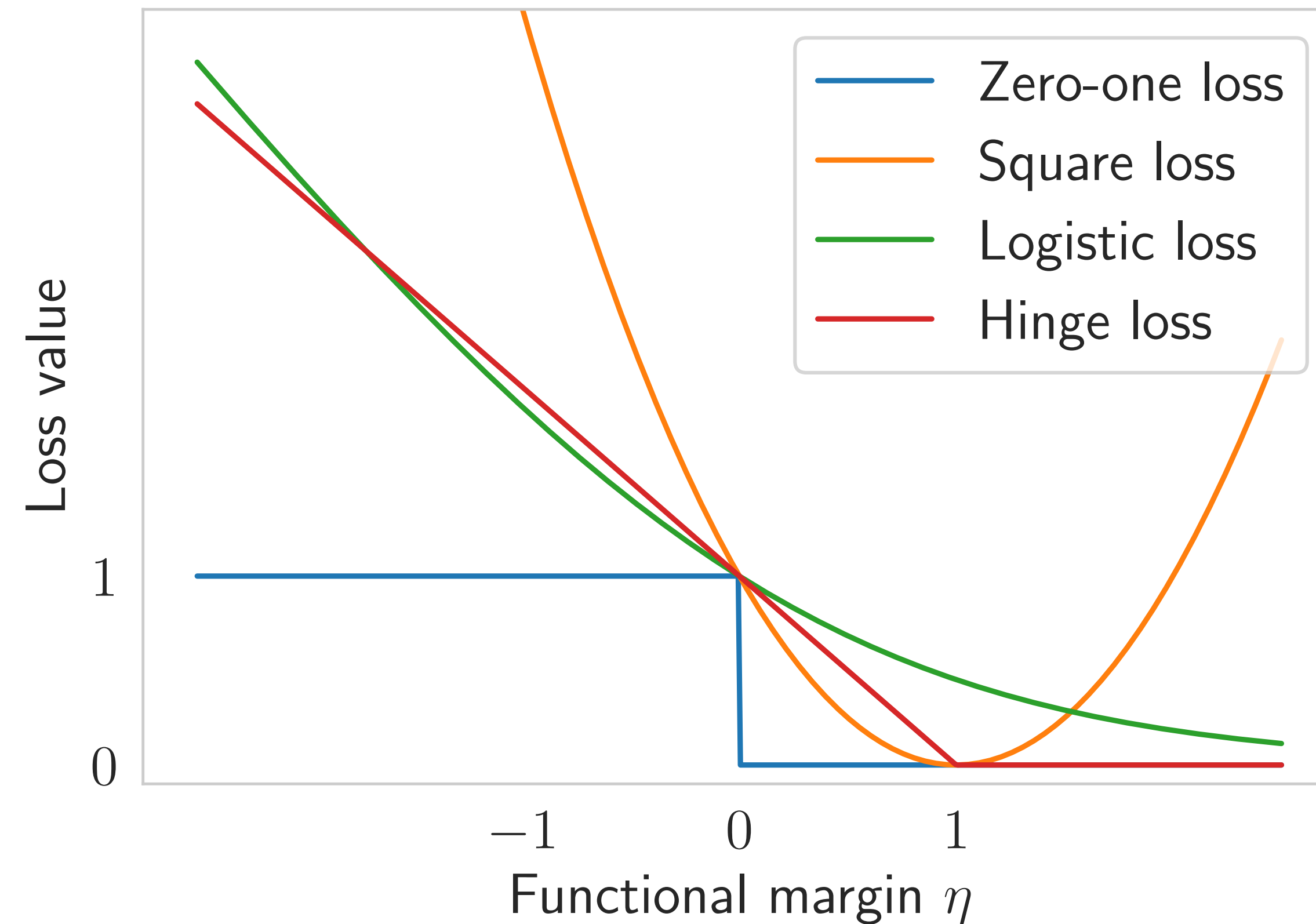
➡ This is a convex problem!



Remark: possible to bound the zero-one risk $L(f)$ by the ϕ risk *

* Under technical assumptions on the function ϕ

Losses for Classification



Logistic loss $\rightarrow$ logistic regression

Hinge loss $\rightarrow$ max margin classification

Do we still have time?

Bonus: a good regressor implies a good classifier

Consider $\mathcal{Y} = \{-1, 1\}$, any regressor $\eta : \mathcal{X} \rightarrow \mathbb{R}$ induces a classifier:

$$f_\eta(x) = \text{sign}(\eta(x))$$

Squared-loss regression

$$L_{\mathcal{D}}^{\ell_2}(\eta) = \mathbb{E}_{\mathcal{D}}[(Y - \eta(X))^2]$$

$$\eta_* \in \arg \min_{\eta} L_{\mathcal{D}}^{\ell_2}(\eta)$$

$$\eta_*(x) = \mathbb{E}[Y | X = x]$$

Classification

$$L_{\mathcal{D}}^{\text{classif}}(f) = \mathbb{E}_{\mathcal{D}}[1_{f(X) \neq Y}]$$

$$f_* \in \arg \min_f L_{\mathcal{D}}^{\text{classif}}(f)$$

We can choose $f_* = f_{\eta_*}$

Claim:

$$L_{\mathcal{D}}^{\text{classif}}(f_\eta) - L_{\mathcal{D}}^{\text{classif}}(f_*) \leq \sqrt{L_{\mathcal{D}}^{\ell_2}(\eta) - L_{\mathcal{D}}^{\ell_2}(\eta_*)}$$

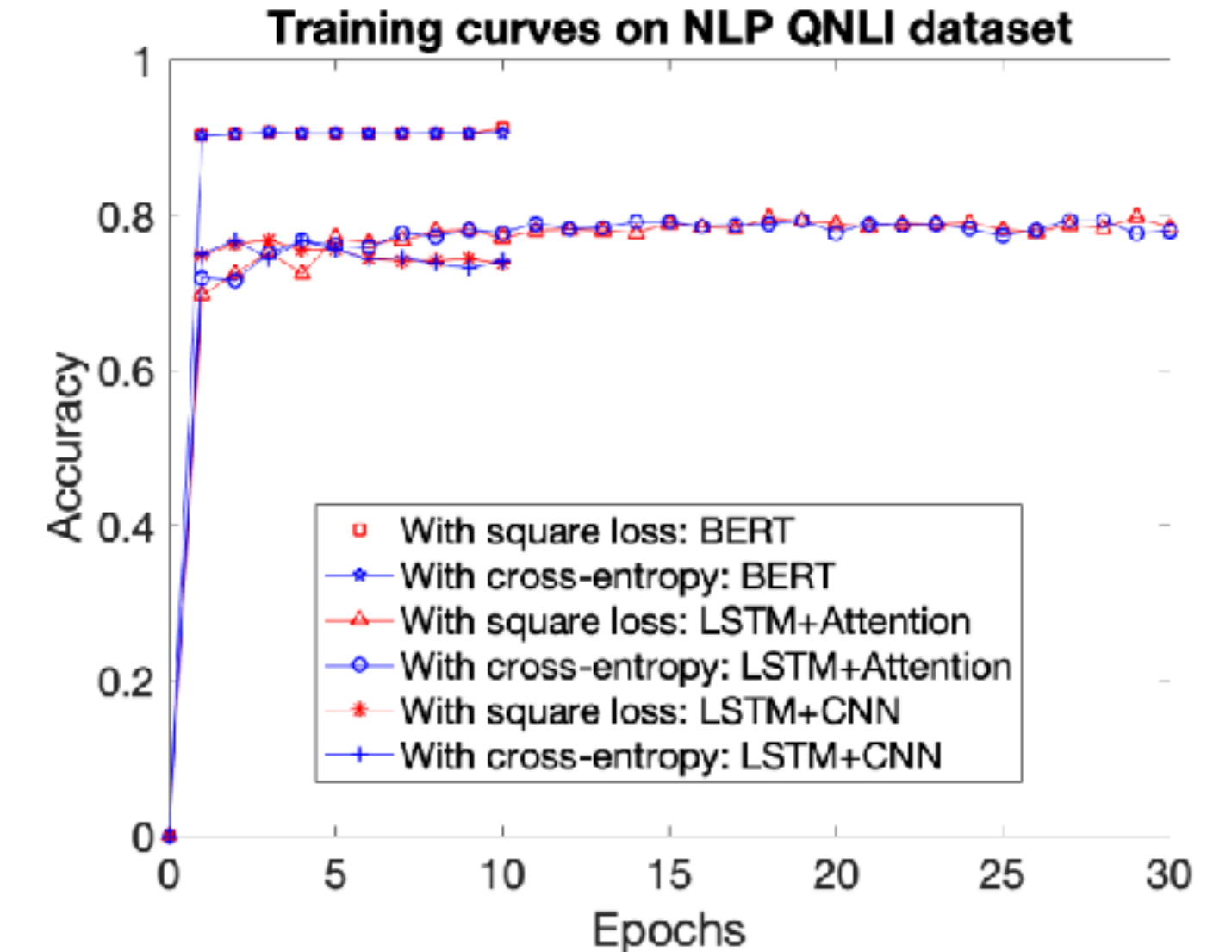
➡ Small excess regression risk $\Rightarrow$ small excess classification risk

Bonus: does the loss function matter? (Over-parameterization regime)

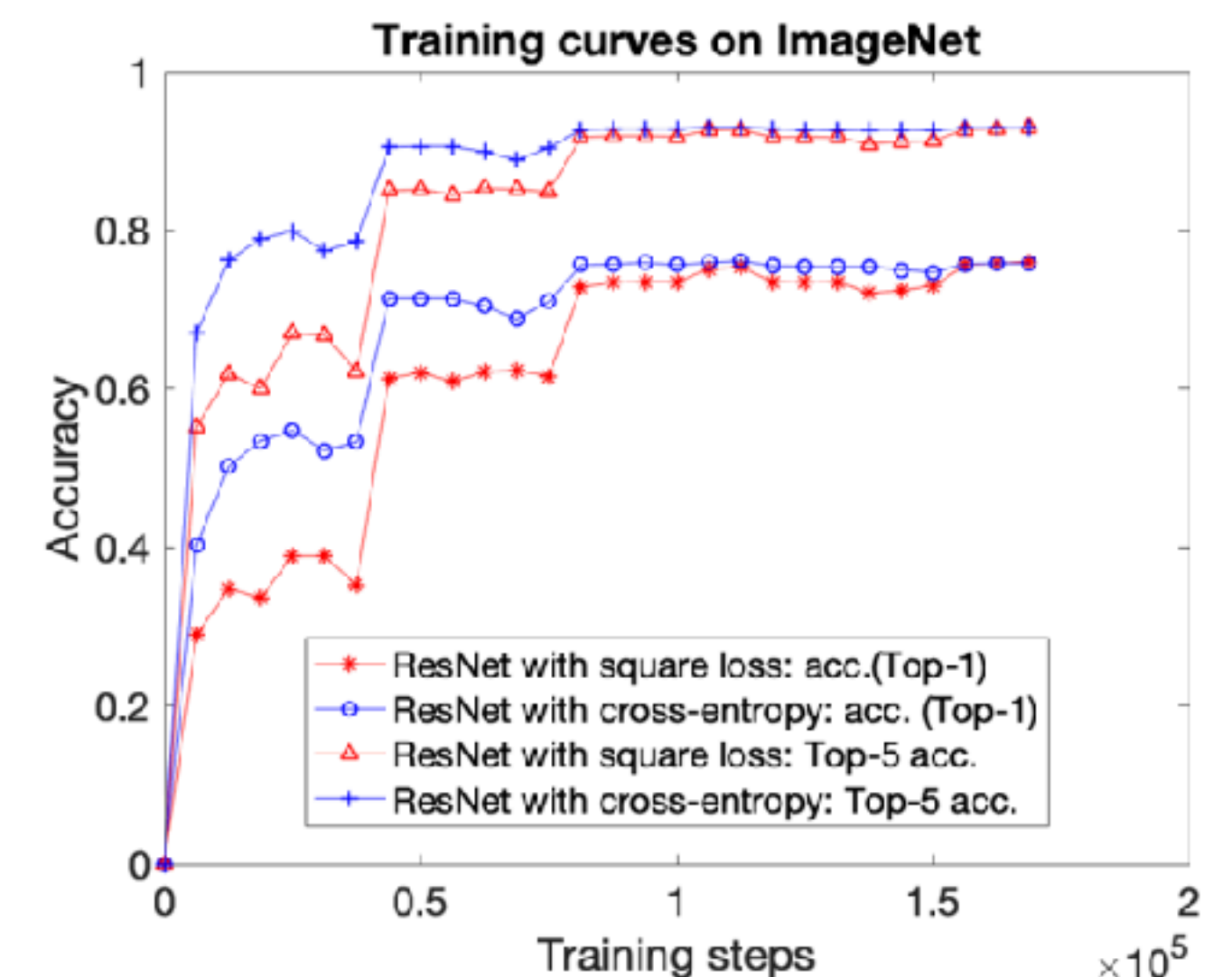
Assume sufficient over-parameterization ($n \ll d$), i.e. all training points are *equally close* to the max-margin separating hyperplane, with high probability

Optimization (training):

the outcome of optimization with *gradient descent*, is the same for both logistic loss and square loss



(a) NLP tasks



(c) Vision tasks

Reference: [Classification vs regression in overparameterized regimes: Does the loss function matter?](#)

Pics from [Evaluation of Neural Architectures Trained with Square Loss vs Cross-Entropy in Classification Tasks](#)

Recap

- Classification:
 - Mapping inputs to discrete outputs (categorical)
 - **Not** a special form of regression!
- Ways to perform classification:
 - Non-parametric: K-Nearest-Neighbors
 - Parametric: learning X-to-Y mapping via ERM
 - More complex decision boundaries? Non-linear classifiers
- Classification vs. Regression:
 - Classical regime: classification tasks cannot be well-solved using regression methods
 - Over-parameterization regime: solutions trained from both regression and classification methods are equivalent